

SahAI

Frontend ↔ Backend

Gap Analysis

A complete comparison of what the app expects and what the API actually provides — before any integration work begins.

REPOSITORY	EdgeNova_SahAI
BRANCH	secure-connect-prototype
COMMIT	3e34621 — “Secure Development”
DATE	27 August 2026
SCOPE	Analysis only — no code modified, committed or pushed
AUDIENCE	Frontend, backend and hardware teammates; team leader; mentor

Contents

1. Executive Summary
2. Current System Architecture
3. Frontend Feature Inventory
4. Backend API Inventory
5. Authentication Comparison
6. Database Schema Summary
7. Frontend ↔ Backend Master Gap Matrix
8. Detailed Data Contract Comparisons
 1. Crop Recommendation
 2. Fertilizer Recommendation
 3. Members
 4. Savings
 5. Transactions
 6. Loans
 7. Loan Risk
 8. Secure Ledger
 9. Assistant
 10. Crop Health / My Land / Reports
9. Frontend-Only Features
10. Backend-Only Features
11. Duplicate Data / Source-of-Truth Problems
12. Integration Blockers
13. Recommended Integration Order
14. Recommended Future Sources of Truth
15. Final Summary
16. Next Stage — Hardware + Software Integration

Branch naming

No branch named exactly `SecureConnectPrototype` exists on the remote. Two branches sit at the **same commit** `3e34621`: `secure-connect-prototype` and `secure_prototype`. Because both point at identical code there is no ambiguity about what was analysed. This report uses `secure-connect-prototype`. **No new branch was created.** The team should agree on one canonical name and delete the duplicate.

1. Executive Summary

The frontend and backend are each individually functional, but **they have never been connected**. This is not a case of small field mismatches on top of a working integration — there is currently no integration at all.

The five findings that matter

#	FINDING	SEVERITY
1	The frontend contains zero HTTP calls . Searching <code>app/</code> , <code>components/</code> and <code>services/</code> for <code>fetch(</code> or <code>axios</code> returns nothing. Every service resolves canned data.	BLOCKER
2	The frontend has no authentication code whatsoever — no login, no token storage, no <code>Authorization</code> header, no 401 handling. Six backend endpoints require a JWT.	BLOCKER
3	The crop model artifacts are not committed . <code>.gitignore</code> excludes <code>models/*.pt</code> and <code>models/*.pkl</code> — exactly what the crop model saves. A fresh clone cannot serve <code>/predict-crop</code> .	BLOCKER
4	Six of the fifteen data-backed screens have no backend endpoint at all — members list, transactions list, loans list, finance summary, crop health, my land.	BLOCKER
5	The endpoint paths configured in the frontend match no backend route , and the port differs from the documented one.	HIGH

What is genuinely in good shape

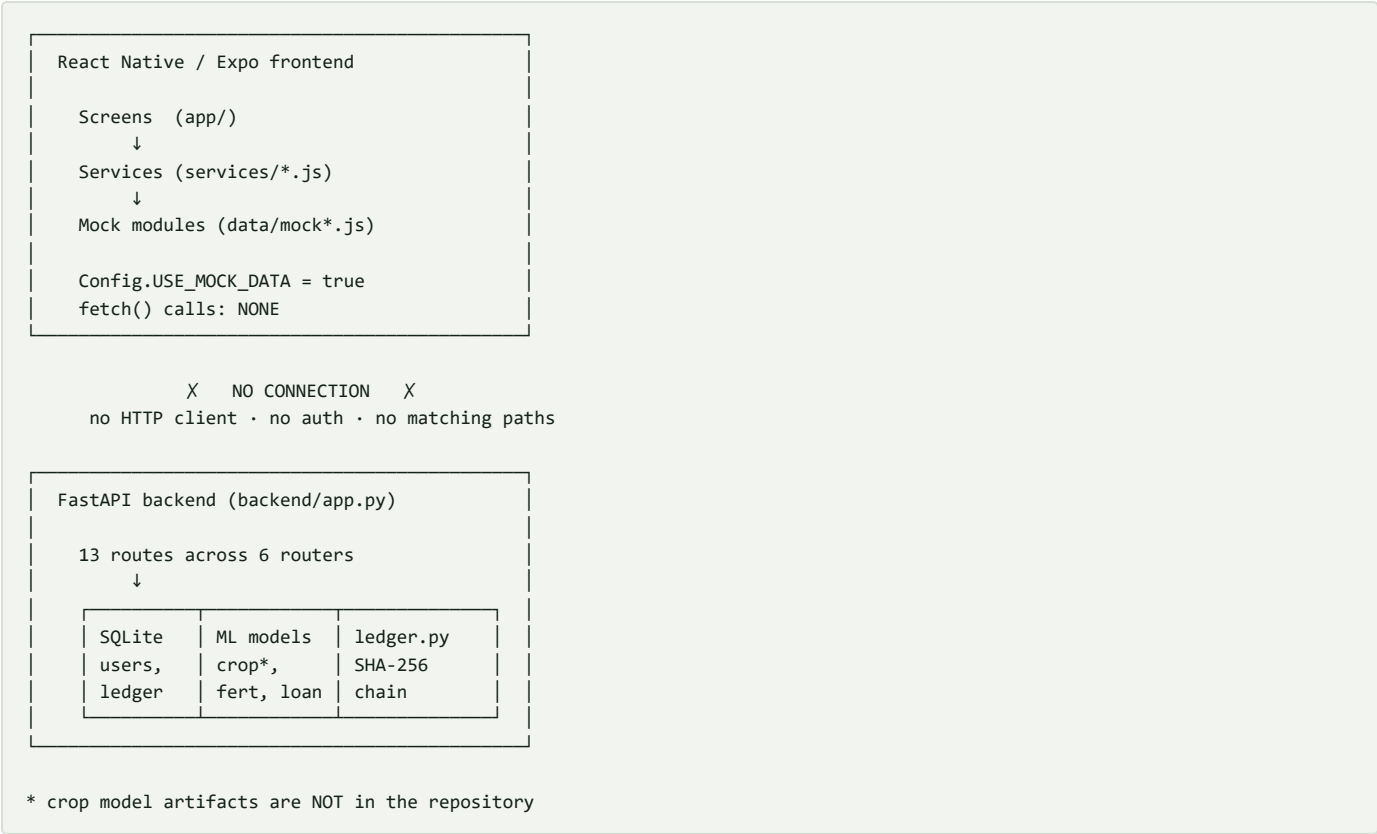
- The **backend is coherent and complete for what it covers**: real PBKDF2 password hashing, real JWT with role-based access, a real SHA-256 hash-chained ledger, and two working ML pipelines with a carefully written crop→fertilizer vocabulary bridge.
- The **frontend is internally consistent**. Its mock modules cross-import from single sources rather than copy-pasting figures, so there is no drift *within* the frontend.
- The **crop recommendation form already matches the model's technical contract** (§8.1). The farmer-friendly redesign this analysis was asked to check for **has not been applied on this branch**.

Scale of the gap

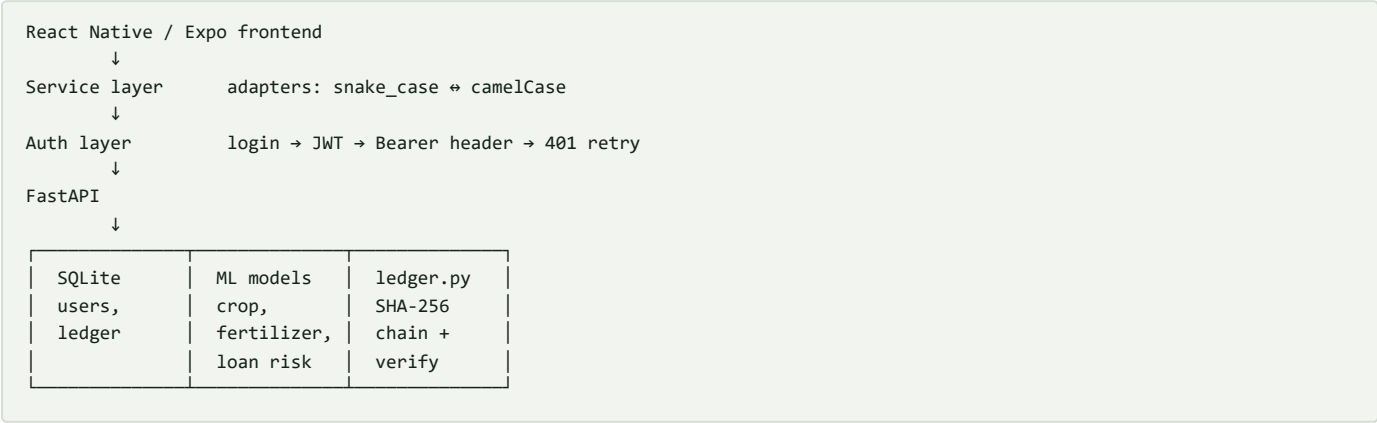
MEASURE	COUNT
Backend endpoints implemented	13
Backend endpoints the frontend is configured to call	4 (all paths wrong)
Backend endpoints the frontend actually calls	0
Frontend screens with a data dependency	15
Frontend screens backed by a real endpoint today	0
Frontend screens for which no backend endpoint exists	6

2. Current System Architecture

As it exists today



Target architecture



Repository structure (actual)

PATH	PRESENT	CONTENTS
frontend/	Yes	Expo app — app/, components/, services/, data/, constants/, utils/
backend/	Yes	FastAPI — app.py, auth.py, ledger.py, models/, routes/, services/
models/	Yes	3 training scripts; fertilizer + loan artifacts committed, crop artifacts not
data/	Yes	raw/README.md and .gitkeep only — no datasets committed
docs/	Yes	.gitkeep only (this report is the first document)
notebooks/	Yes	.gitkeep only
hardware/	No	Does not exist. No ESP32 or IoT code anywhere in the repository.

3. Frontend Feature Inventory

Data source per screen, read from the actual imports on this branch.

SCREEN	ROUTE FILE	SERVICE USED	CURRENT DATA SOURCE
Home	app/(tabs)/home.jsx	—	mockHomeData (hardcoded)
Farm hub	app/(tabs)/farm.jsx	—	Static navigation — no data dependency
Finance dashboard	app/(tabs)/finance.jsx	financeService	Mock service
Assistant	app/(tabs)/assistant.jsx	assistantService	Mock service + mockAssistantData
Crop Recommendation	app/crop-recommendation.jsx	cropService	Mock service
Fertilizer Advice	app/fertilizer-advice.jsx	fertilizerService	Mock service
Loan Risk	app/loan-risk.jsx	loanService, memberService	Mock service
Secure Ledger	app/ledger.jsx	ledgerService	Mock service
Ledger Record	app/ledger-record.jsx	ledgerService	Mock service + mockLedgerData
Transactions	app/transactions.jsx	financeService	Mock service + mockFinanceData
Members	app/members.jsx	memberService	Mock service (module-level array)
Member Details	app/member-details.jsx	memberService	Mock service + mockFinanceData
Add Member	app/add-member.jsx	memberService	Local state only — lost on reload
Savings	app/savings.jsx	financeService, memberService	Mock service; write is a no-op
Loans	app/loans.jsx	financeService	Mock service
My Land	app/my-land.jsx	—	mockLandData (hardcoded)
Crop Health	app/crop-health.jsx	—	mockCropHealthData (hardcoded)
Reports	app/reports.jsx	—	mockReportData (hardcoded)
Authentication	—	—	Not implemented in the frontend

Frontend configuration

```
// constants/Config.js
API_BASE_URL: 'http://localhost:8000'
USE MOCK_DATA: true

ENDPOINTS: {
  cropRecommendation: '/predict/crop',
  fertilizerRecommendation: '/predict/fertilizer',
  loanRisk: '/predict/loan-risk',
  ledgerVerify: '/ledger/verify',
}
```

Three of those four paths do not exist on the backend. See §7.

4. Backend API Inventory

All 13 endpoints, read from `backend/routes/`.

#	METHOD	PATH	AUTH	REQUEST FIELDS	KEY RESPONSE FIELDS
1	POST	/auth/register	None	member_id, name, password, role	member_id, role, token
2	POST	/auth/login	None	member_id, password	member_id, role, token
3	GET	/auth/me	Any authed	—	member_id, role, exp
4	POST	/predict-crop	None	N, P, K, temperature, humidity, ph, rainfall	recommended_crop, confidence, alternatives[], fertilizer_available
5	POST	/crop-advisory	None	crop fields + soil_type, moisture, fert_nitrogen, fert_potassium, fert_phosphorous	crop{}, mapped_crop_type, fertilizer{}, needs_soil_test
6	GET	/fertilizer/options	None	—	soil_types[], crop_types[], fertilizer_classes[], guideline_only_crops[]
7	POST	/recommend-fertilizer	None	temperature, humidity, moisture, nitrogen, potassium, phosphorous, soil_type, crop_type	recommended_fertilizer, confidence, (source, note, soil_note)
8	POST	/request-loan	Any authed	amount, term_in_months, sector, repayment_interval, member_id?	risk_score, risk_label, flagged_high_risk, decision_threshold, savings_consistency(+detail), request, model, note
9	POST	/ledger/add	Any authed (self; treasurer for others)	member_id, entry_type, amount	id, member_id, entry_type, amount, timestamp, entry_hash
10	GET	/ledger/verify	Treasurer/admin	—	valid, broken_entry_id
11	GET	/ledger/all	Treasurer/admin	—	array of raw ledger rows
12	GET	/member/{id}/portfolio	Self, or treasurer/admin	path param	member_id, total_savings, total_loans_outstanding, net_position, history[]
13	GET	/group/summary	Treasurer/admin	—	member_count, total_corpus, total_outstanding_loans, members[]

Startup: `uvicorn backend.app:app --reload --port 5000` (per the `backend/app.py` docstring). **CORS:** `allow_origins=["*"]`, all methods and headers — permissive enough for development.

Endpoints that do NOT exist

NEEDED BY	MISSING ENDPOINT
Members screen	GET members list
Add Member screen	POST create member (only <code>/auth/register</code> , which takes no phone/village)
Transactions screen	GET transactions list (only treasurer-gated <code>/ledger/all</code>)
Loans screen	GET loans list
Finance dashboard	GET finance summary in the frontend's shape
Ledger Record screen	GET single ledger record by id
Any client	GET <code>/health</code>
Crop Health, My Land	anything at all

5. Authentication Comparison

ASPECT	BACKEND	FRONTEND
Password hashing	PBKDF2-SHA256, per-user salt, 100,000 iterations	n/a
Token type	JWT, HS256	Not implemented
Secret	SAHAI_SECRET_KEY env var, with a dev fallback default	n/a
Login endpoint	POST /auth/login → {member_id, role, token}	No login screen, no login call
Header scheme	Authorization: Bearer <token> via HTTPBearer	Never sent
Expiry	30 minutes (TOKEN_EXPIRY_SECONDS = 60 * 30)	No refresh logic
401 behaviour	“Token expired, please log in again.” / “Invalid token.”	No handler
403 behaviour	“This action requires treasurer or admin access.”	No handler
Roles	admin, treasurer, member; staff = {admin, treasurer}	No role concept
Token storage	n/a	Nowhere

Verdict: BLOCKER

Six endpoints — /request-loan, /ledger/add, /ledger/verify, /ledger/all, /member/{id}/portfolio and /group/summary — are unreachable from the frontend as it stands. The frontend has no way to obtain, store or send a token, and no screen exists for a user to enter credentials.

A further design question the team must answer: **who is the app logged in as?** The ledger and portfolio endpoints are scoped per member, and verification is treasurer-only. The frontend currently assumes a single anonymous operator.

6. Database Schema Summary

SQLite at backend/database.db. Two tables, both created on startup.

users

COLUMN	TYPE	CONSTRAINTS
member_id	TEXT	PRIMARY KEY
name	TEXT	NOT NULL
password_hash	TEXT	NOT NULL
role	TEXT	NOT NULL, CHECK IN (member, treasurer, admin)

ledger

COLUMN	TYPE	CONSTRAINTS
id	INTEGER	PRIMARY KEY AUTOINCREMENT
member_id	TEXT	NOT NULL (no FK constraint declared)
entry_type	TEXT	NOT NULL — savings_deposit loan_disbursed loan_repayment
amount	REAL	NOT NULL
timestamp	REAL	NOT NULL (epoch seconds)
prev_hash	TEXT	NOT NULL
entry_hash	TEXT	NOT NULL

Tables that do not exist

`members` (as a profile table), `transactions`, `loans`, `savings`, `land`, `crop_health`, `reports`.

Everything financial is derived from the `ledger` table, not stored:

CONCEPT	HOW THE BACKEND PRODUCES IT
Member savings	Sum of <code>savings_deposit</code> rows for that member
Outstanding loan	Sum of <code>loan_disbursed</code> – sum of <code>loan_repayment</code>
Group corpus	Sum of all <code>savings_deposit</code> rows
Member list	Distinct <code>member_id</code> values in <code>ledger</code> — not from <code>users</code>

Two additional notes

1. `DB_PATH = "backend/database.db"` is a **relative** path in both `backend/ledger.py` and `backend/models/user.py`. Started from any directory other than the repository root, SQLite silently creates a *new, empty* database instead of failing.
2. The database is gitignored. A fresh clone starts with **zero rows**, so every finance screen would legitimately render empty even after integration.

7. Frontend ↔ Backend Master Gap Matrix

Split across two tables so every required column stays legible in print. Both share the *Feature* column.

7A — Contract and endpoint

FEATURE	FRONTEND FILE	SERVICE	BACKEND ENDPOINT	AUTH?	FE HAS, BE MISSING	BE HAS, FE MISSING
Authentication	(none)	—	/auth/login, /register, /me	—	Login UI, token store, Bearer header, 401 retry	Roles, 30-min expiry, register
Crop Recommendation	crop-recommendation.jsx	cropService	POST /predict-crop	No	message	alternatives[], fertilizer_available
Fertilizer Advice	fertilizer-advice.jsx	fertilizerService	POST /recommend-fertilizer	No	quantity, quantityUnit, estimatedSaving, actions[]	temperature, humidity, moisture, soil_type (all required), confidence, source
Loan Risk	loan-risk.jsx	loanService	POST /request-loan	Yes	repaymentProbability, recommendation, reasons[]	flagged_high_risk, decision_threshold, savings_consistency_detail, model{}
Members list	members.jsx	memberService	none	—	Whole feature	—
Member details	member-details.jsx	memberService	GET /member/{id}/portfolio	Yes	name, phone, village, joinedAt, repaymentStatus	net_position
Add Member	add-member.jsx	memberService	POST /auth/register (partial)	No	phone, village, initialSavings	password, role
Savings	savings.jsx	financeService	POST /ledger/add (write)	Yes	Savings totals endpoint, per-member breakdown	Hash-chained write
Transactions	transactions.jsx	financeService	GET /ledger/all (treasurer)	Yes	description, status, member name, expense type	prev_hash, entry_hash
Loans	loans.jsx	financeService	none	—	Whole feature: durationMonths, purpose, status	—
Finance dashboard	(tabs)/finance.jsx	financeService	GET /group/summary (partial)	Yes	availableBalance, savingsThisMonth, savingsDelta	members[]
Secure Ledger	ledger.jsx	ledgerService	GET /ledger/all + /verify	Yes	memberName, note, direction, per-record verified	—
Ledger Record	ledger-record.jsx	ledgerService	none	—	Single-record lookup	—
Home dashboard	(tabs)/home.jsx	—	none	—	Every metric	—
Assistant	(tabs)/assistant.jsx	assistantService	none	—	All answers	—
Reports	reports.jsx	—	none	—	Every figure	—
Crop Health	crop-health.jsx	—	none	—	Whole feature	—
My Land	my-land.jsx	—	none	—	Whole feature	—

7B — Mismatch, status and resolution

FEATURE	NAMING / TYPE MISMATCH	CURRENT STATUS	SEVERITY	RECOMMENDED RESOLUTION
Authentication	—	Not connected	BLOCKER	Add an auth layer before anything else
Crop Recommendation	nitrogen→N, phosphorus→P, potassium→K; confidence 0-100 int vs 0-1 float; factors[] vs alternatives[]	Mock only	BLOCKER	Wire it; rename 3 fields; scale confidence ×100

FEATURE	NAMING / TYPE MISMATCH	CURRENT STATUS	SEVERITY	RECOMMENDED RESOLUTION
Fertilizer Advice	phosphorus→phosphorous; frontend ph not accepted	Mock only; form cannot satisfy the API	BLOCKER	Add 4 missing inputs; drop quantity / estimatedSaving
Loan Risk	requestedAmount→amount; durationMonths→term_in_months; score 0-100 vs 0-1; Low vs LOW	Mock only	BLOCKER	Needs auth first; 3 frontend inputs have no backend feature
Members list	—	Mock only	BLOCKER	New endpoint required
Member details	savings→total_savings; outstandingLoan→total_loans_outstanding	Mock only	HIGH	Extend portfolio, or new endpoint
Add Member	—	Local state only	BLOCKER	Backend cannot store phone/village — schema change needed
Savings	type: 'savings' → entry_type: 'savings_deposit'	Mock; write is a no-op	BLOCKER	Wire write to /ledger/add; add read endpoint
Transactions	type enum differs on all values; date display string vs epoch float	Mock only	BLOCKER	New non-treasurer endpoint; enum + name mapping
Loans	—	Mock only	BLOCKER	Derive from ledger, or add a loans table
Finance dashboard	totalSavings→total_corpus; activeMembers→member_count; outstandingLoans→total_outstanding_loans	Mock only	HIGH	Extend /group/summary or add a summary endpoint
Secure Ledger	entry_type vs type / kind; prev_hash→previousHash; epoch vs ISO; valid→verified	Mock only	BLOCKER	Wire both; adapt shapes in the service
Ledger Record	—	Mock only	MEDIUM	Filter client-side from /ledger/all, or add endpoint
Home dashboard	—	Hardcoded	HIGH	Compose from other services once they work
Assistant	—	Mock only	HIGH	Re-point answers at real services
Reports	—	Hardcoded	MEDIUM	Derive from real services
Crop Health	—	Hardcoded	LOW	No backend exists — decide scope
My Land	—	Hardcoded	LOW	No backend exists — decide scope

Endpoint path mismatches

CONFIG.ENDPOINTS VALUE	ACTUAL BACKEND ROUTE	MATCH?
/predict/crop	/predict-crop	No
/predict/fertilizer	/recommend-fertilizer	No
/predict/loan-risk	/request-loan	No
/ledger/verify	/ledger/verify	Yes

API_BASE_URL is `http://localhost:8000`; `backend/app.py` documents `--port 5000`. **Port mismatch — severity HIGH**. Also note that on a physical phone `localhost` resolves to the phone itself, so a LAN address will be needed for device testing.

8. Detailed Data Contract Comparisons

8.1 Crop Recommendation

Request

FRONTEND FIELD	FE TYPE	REQ?	BACKEND FIELD	BE TYPE	REQ?	MATCH?	PROBLEM
nitrogen	number	Yes	N	float	Yes	Rename	Naming only
phosphorus	number	Yes	P	float	Yes	Rename	Naming only
potassium	number	Yes	K	float	Yes	Rename	Naming only
temperature	number	Yes	temperature	float	Yes	Yes	—
humidity	number	Yes	humidity	float	Yes	Yes	—
ph	number	Yes	ph	float	Yes	Yes	—
rainfall	number	Yes	rainfall	float	Yes	Yes	—

Response

FRONTEND EXPECTS	FE TYPE	BACKEND RETURNS	BE TYPE	MATCH?	PROBLEM
crop	string "Rice"	recommended_crop	string "rice"	Rename	Also lower-case vs title-case
confidence	int 94	confidence	float 0.8674	No	Scale differs — 0-100 vs 0-1
message	string	—	—	No	Frontend-only prose
factors[]	string[]	alternatives[]	[(crop, confidence)]	No	Different meaning and shape
—	—	fertilizer_available	bool	—	Backend-only, unused

§11 finding — the hypothesised farmer-friendly mismatch does NOT exist on this branch

The brief asked whether the frontend now collects *District*, *Soil Type*, *Growing Time*, *Water Availability*, *Watering Method*, *Previous Crop*, *Land Size*. It does **not**. The form on `secure-connect-prototype` collects the seven technical values above, which align with the trained model almost exactly. If a farmer-friendly redesign exists, it is on another branch and was not analysed here.

Consequence: crop recommendation is one of the *easiest* integrations, not one of the hardest — three renames and one ×100 scaling.

Blocking caveat

The crop model artifacts (`crop_model.pt`, `crop_scaler.pkl`, `crop_label_encoder.pkl`) are **not committed**, because `.gitignore` excludes `models/*.pt` and `models/*.pkl`. On a fresh clone this endpoint returns 503 "Crop model not trained yet." regardless of any frontend work.

8.2 Fertilizer Recommendation

Request

FRONTEND FIELD	FE TYPE	REQ?	BACKEND FIELD	BE TYPE	REQ?	MATCH?	PROBLEM
crop	string	Yes	crop_type	string	Yes	Rename	Backend maps crop-model names itself
nitrogen	number	Yes	nitrogen	float	Yes	Yes	—
potassium	number	Yes	potassium	float	Yes	Yes	—
phosphorus	number	Yes	phosphorous	float	Yes	Rename	Different spelling
ph	number	Yes	—	—	—	No	Backend does not use pH at all
—	—	—	temperature	float	Yes	No	Not collected by the form
—	—	—	humidity	float	Yes	No	Not collected by the form
—	—	—	moisture	float	Yes	No	Not collected by the form
—	—	—	soil_type	string	Yes	No	Not collected by the form

Response

FRONTEND EXPECTS	FE TYPE	BACKEND RETURNS	MATCH?	PROBLEM
fertilizer	string	recommended_fertilizer	Rename	—
quantity	number 35	—	No	Backend has no dosage output
quantityUnit	string kg/ha	—	No	Backend has no dosage output
estimatedSaving	number 1200	—	No	Backend has no savings output
message	string	—	No	Frontend-only prose
actions[]	string[]	note, soil_note	Partial	Guideline path only; different shape
—	—	confidence	—	Backend-only, unused
—	—	source	—	model guideline_table — backend-only

\$12 finding — the worst contract mismatch in the project

The form is missing **four required backend fields**, so a request built from it would be rejected with 422 before reaching the model. Additionally, three of the values the result screen displays (quantity, quantityUnit, estimatedSaving) have **no backend source at all** — the model returns a fertilizer *grade* only.

Known backend vocabularies (from GET /fertilizer/options): soil types Black, Clayey, Loamy, Red, Sandy; crop types Barley, Cotton, Ground Nuts, Maize, Millets, Oil seeds, Paddy, Pulses, Sugarcane, Tobacco, Wheat. The frontend's quick-pick chips are Rice, Wheat, Maize, Cotton, Sugarcane — **Rice is not a backend crop type** (the backend maps rice → Paddy internally, so this one happens to work, but only via that bridge).

No conversion from pH to the missing fields exists or should be invented.

8.3 Members

FRONTEND FIELD	FE TYPE	BACKEND SOURCE	BE TYPE	MATCH?	PROBLEM
id	string MEM-001	users.member_id	TEXT	Rename	—
name	string	users.name	TEXT	Yes	—
phone	string	—	—	No	No column exists
village	string	—	—	No	No column exists
savings	number	derived from ledger	REAL	Derived	Not a stored column
outstandingLoan	number	derived from ledger	REAL	Derived	Not a stored column
repaymentStatus	string "On Track"	—	—	No	No backend concept
joinedAt	date string	—	—	No	No column exists
—	—	users.role	TEXT	—	Backend-only, unused

Endpoint: none for a members list. `build_group_summary()` returns `members[]` as **member IDs only**, derived from the `ledger` table — so a registered member with no transactions would not appear.

8.4 Savings

QUESTION	FRONTEND	BACKEND
Representation	A <code>savings</code> number per member, plus <code>financeSummary.totalSavings</code>	Derived by summing <code>savings_deposit</code> ledger rows
Storage	Mock array in <code>data/mockMembers.js</code>	No savings table, no savings column
Write path	<code>recordSavings()</code> returns a fabricated object; nothing persists	<code>POST /ledger/add</code> writes a real hash-chained row
Monthly figure	<code>savingsThisMonth: 7500</code> hardcoded	Not implemented — no endpoint computes this
Trend / delta	<code>savingsDelta: '+8.2%'</code> hardcoded	Not implemented

Verdict: the models are compatible in principle — both treat savings as a sum of deposits — but the frontend has no way to read or write them today, and two displayed figures have no backend equivalent.

8.5 Transactions

FRONTEND FIELD	FE TYPE	BACKEND FIELD	BE TYPE	MATCH?	PROBLEM
id	string TXN-001	id	INTEGER	No	String vs integer
type	enum (below)	entry_type	enum (below)	No	Every value differs
description	string	—	—	No	Not stored
member	name string	member_id	TEXT	No	Needs a join the API does not do
amount	number	amount	REAL	Yes	—
date	display string “Today”	timestamp	REAL epoch	No	Formatting vs raw
status	“Completed”	—	—	No	No backend concept
statusTone	“success”	—	—	No	Presentation only
—	—	prev_hash, entry_hash	TEXT	—	Backend-only, unused

Enum mismatch — every value differs

FRONTEND TYPE	BACKEND ENTRY_TYPE	MATCH?
savings	savings_deposit	No
disbursement	loan_disbursed	No
repayment	loan_repayment	No
expense	(no equivalent)	No — backend cannot represent this

The Transactions screen also offers an **Expenses** filter, which no backend entry type can ever populate.

8.6 Loans

FRONTEND FIELD	FE TYPE	BACKEND EQUIVALENT	MATCH?	PROBLEM
id	string LN-001	—	No	No loan entity
memberId	string	ledger.member_id	Partial	Only via ledger rows
member	name string	—	No	Requires a join
principal	number	sum of loan_disbursed	Derived	Not stored
remaining	number	disbursed – repaid	Derived	Not stored
durationMonths	number 12	—	No	Not recorded anywhere
status	“On Track” / “Delayed”	—	No	No backend concept
purpose	“Crop inputs”	—	No	Not recorded anywhere

There is no loans table and no loans endpoint. A loan exists only as a pair of ledger entry types. Loan *term* and *purpose* are captured nowhere, even though `/request-loan` accepts `term_in_months` as an input — it is used for scoring and then discarded, not persisted.

8.7 Loan Risk

Request

FRONTEND FIELD	FE TYPE	REQ?	BACKEND FIELD	BE TYPE	REQ?	MATCH?	PROBLEM
requestedAmount	number (₹)	Yes	amount	float (₹)	Yes	Rename	Backend converts ₹→USD at <code>INR_PER_USD = 83.0</code>
durationMonths	number	Yes	term_in_months	int	Yes	Rename	—
memberId	string	Yes	member_id	string	Optional	Rename	Treasurer-only for other members
monthlyIncome	number	Yes	—	—	—	No	Model has no income feature
existingLoan	number	Yes	—	—	—	No	Model has no existing-debt feature
repaymentScore	number (%)	Yes	—	—	—	No	Backend derives <code>savings_consistency</code> from the ledger instead
—	—	—	sector	string	Optional	No	Not collected (defaults Agriculture)
—	—	—	repayment_interval	string	Optional	No	Not collected (defaults monthly)

Response

FRONTEND EXPECTS	FE TYPE	BACKEND RETURNS	BE TYPE	MATCH?	PROBLEM
riskLevel	Low/Medium/High	risk_label	LOW/MEDIUM/HIGH	No	Case differs
riskScore	int 18 (%)	risk_score	float 0.009	No	Scale differs
repaymentProbability	int 82	—	—	No	Derivable as 1 – risk_score
recommendation	string	—	—	No	Frontend-only prose
reasons[]	string[]	—	—	No	Model returns no attribution
—	—	flagged_high_risk, decision_threshold	bool, float	—	Backend-only
—	—	savings_consistency (+detail)	float, object	—	The product's key differentiator, unused
—	—	model.label_is_synthetic	bool	—	Honesty flag the UI should surface

Auth required — and a philosophical conflict

This endpoint is unreachable until the frontend can log in.

The backend deliberately *refuses* to accept `savings_consistency` from the client and computes it from the member's own hash-chained deposit history, so a member cannot inflate her own score. The frontend's `repaymentScore` input is therefore not just unmapped — it is *philosophically opposed* to how the backend scores. The team should decide whether to remove that field.

8.8 Secure Ledger

Record shape

FRONTEND FIELD	FE TYPE	BACKEND FIELD	BE TYPE	MATCH?	PROBLEM
id	TXN-024	id	INTEGER	No	String vs int
sequence	number	id	INTEGER	Partial	Same value, different name
memberName	string	member_id	TEXT	No	Backend returns no name
type	“Savings Deposit”	entry_type	savings_deposit	No	Display vs enum
kind	“savings”	entry_type	savings_deposit	No	Second enum, also differs
amount	number	amount	REAL	Yes	—
direction	in/out	—	—	No	Derivable from entry_type
timestamp	ISO string	timestamp	REAL epoch	No	Format differs
note	string	—	—	No	Not stored
verified	bool per record	—	—	No	Backend verifies the whole chain , not rows
currentHash	string	entry_hash	TEXT	Rename	—
previousHash	string	prev_hash	TEXT	Rename	—

Verification

FRONTEND EXPECTS	BACKEND RETURNS	MATCH?
verified	valid	Rename
totalRecords	—	No — count client-side from /ledger/all
checkedRecords	—	No
tamperedRecordId (TXN-017)	broken_entry_id (int)	Rename + type
verifiedAt	—	No

Endpoints: GET /ledger/all and GET /ledger/verify both require **treasurer or admin**. There is **no single-record endpoint**, so ledger-record.jsx has no backend source. The frontend's ?demo=tampered / empty / error modes simulate states the backend would otherwise report.

8.9 Assistant

QUESTION	ANSWER
Uses a language model?	No — and none should be added
How it works	Keyword matcher over intentKeywords, returning entries from a fixed assistantResponses table
Data source	data/mockAssistantData.js
Can it reflect real backend data today?	No

mockAssistantData.js imports from cropService, fertilizerService, loanService, mockFinanceData and mockLedgerData — so its answers are consistent *with the mocks*. Once those services return real data, the Assistant's figures would follow **only if** the answers are rebuilt at query time; today they are assembled once at module load, which would freeze whatever the first values were. **No backend chat endpoint exists.**

8.10 Crop Health / My Land / Reports

FEATURE	BACKEND SUPPORT	CURRENT SOURCE	NOTES
Crop Health	None — no table, no route	mockCropHealthData.js	Health %, plot scores and issues have no backend concept
My Land	None — no table, no route	mockLandData.js	Farm name, acreage, plots, soil type, irrigation all frontend-only
Reports	Partial (indirect)	mockReportData.js	Fund half is derivable from the ledger; field half depends on the two above, so it is not derivable

None of these three will work after backend integration without new backend work or a decision to descope them.

9. Frontend Features With No Backend Support

FEATURE	NATURE	NOTES
Crop Health	mock-only	No table, no endpoint
My Land	mock-only	No table, no endpoint
Reports (field half)	mock-only	Depends on the two above
Home dashboard	hardcoded	Every metric; composable once other services work
Assistant answers	mock-only	No backend endpoint
Add Member — phone, village, initialSavings	local-state-only	Lost on reload; backend has no columns for phone/village
Loans list — durationMonths, purpose, status	mock-only	No loans table
Transaction expense type	mock-only	Backend enum cannot represent it
Transaction status / statusTone	UI-only	Presentation, not data — fine to keep
Ledger note field	mock-only	Not stored
Ledger per-record verified	mock-only	Backend verifies the whole chain
Fertilizer quantity, estimatedSaving	mock-only	Model outputs a grade only
Loan risk reasons[], recommendation	mock-only	Model gives no attribution
savingsThisMonth, savingsDelta	mock-only	No endpoint computes either
Authentication UI	not implemented	Backend is ready and waiting

10. Backend Features Not Yet Used By The Frontend

Capabilities that already exist and need **no backend work** — only frontend wiring.

BACKEND CAPABILITY	ENDPOINT	WHY IT MATTERS
User registration and login	/auth/register, /auth/login	Complete JWT auth is already built
Role-based access control	require_treasurer, require_admin	Treasurer vs member views are already enforceable
Current-user lookup	/auth/me	—
Crop→fertilizer advisory chain	POST /crop-advisory	One call does crop and fertilizer, including the vocabulary bridge
Fertilizer vocabularies	GET /fertilizer/options	Would let the form populate real dropdowns instead of hardcoded chips
Guideline fertilizer table	inside /recommend-fertilizer	Covers 10 fruit/plantation crops the model cannot handle
Ranked crop alternatives	alternatives[]	Richer than the frontend's <code>factors[]</code>
Ledger-derived savings consistency	inside /request-loan	The product's core differentiator — currently invisible in the UI
Model honesty metadata	model.label_is_synthetic, val_auc	Should be surfaced to the treasurer
Member portfolio	GET /member/{id}/portfolio	Covers much of Member Details
Group summary	GET /group/summary	Covers part of the Finance dashboard
Hash-chained writes	POST /ledger/add	Every savings/loan write should go through this

11. Duplicate Data / Source-of-Truth Problems

Important nuance

The frontend mocks are **not** copy-pasted. They cross-import from single sources — `mockHomeData` imports `financeSummary` from `mockFinanceData`, `mockCropHealthData` imports `MOCK_RECOMMENDATION` from `cropService`, and so on. The frontend is internally consistent by construction. The duplication problem is therefore **across the frontend/backend boundary**, not within the frontend.

VALUE	FRONTEND SOURCE	BACKEND SOURCE	RISK
Group savings total	<code>financeSummary.totalSavings</code> = 48500	<code>total_corpus</code> (sum of ledger rows)	Two authorities; will disagree the moment the DB has different rows
Outstanding loans	<code>financeSummary.outstandingLoans</code> = 18000	<code>total_outstanding_loans</code> (derived)	Same
Member count	<code>financeSummary.activeMembers</code> = 12	<code>member_count</code> (distinct ledger member IDs)	Definitions differ — frontend counts the roster, backend counts members <i>with transactions</i>
Member savings	<code>mockMembers[].savings</code>	Derived from ledger	Same
Crop recommendation	<code>MOCK_RECOMMENDATION</code> = Rice, 94%	Model output (e.g. rice, 0.8674)	Frontend value is fixed; model value varies per input
Ledger record count	<code>ledgerSummary.totalRecords</code> = 24	<code>COUNT(*)</code> on ledger	Same
Ledger verified state	<code>verifiedResult</code> (always true)	<code>verify_chain()</code>	Frontend can display VERIFIED without the backend having checked anything

The most serious duplication risk is the last row. The ledger's entire value proposition is that verification is real. A frontend that can render "VERIFIED" from a constant undermines that claim. After integration, that badge must come only from `GET /ledger/verify`.

12. Integration Blockers

Issues that genuinely prevent integration, in order of severity.

B1 — The frontend has no HTTP client BLOCKER

There is not a single `fetch()` or `axios` call anywhere in `app/`, `components/` or `services/`. `Config.USE MOCK_DATA` is `true` and no service branches on it to make a real request. Nothing can be integrated until an HTTP layer exists.

B2 — The frontend cannot authenticate BLOCKER

No login screen, no credential entry, no token storage, no `Authorization` header, no 401 handling. Six endpoints are gated. Until this is solved, Loan Risk, Ledger, Transactions, Savings writes, Member Portfolio and Group Summary are all unreachable. The team must also decide **who the app is logged in as**, since ledger reads and verification are treasurer-only.

B3 — Crop model artifacts are not in the repository BLOCKER

`.gitignore` excludes `models/*.pt`, `models/*.pkl` and `models/*.json`. The crop training script saves exactly `crop_model.pt`, `crop_scaler.pkl` and `crop_label_encoder.pkl`. The fertilizer and loan models survived only because they use `.pth` and `.joblib`, which the rule does not cover. On a fresh clone `/predict-crop` and `/crop-advisory` both return 503.

Also: no datasets are committed (`data/raw/*.csv` is gitignored), so the model cannot simply be retrained from the repository either.

B4 — Six screens have no backend endpoint BLOCKER

Members list, Add Member (with phone/village), Transactions list, Loans list, Crop Health and My Land. These require **new backend work**, not frontend wiring.

B5 — The fertilizer form cannot satisfy its own API BLOCKER

The form collects `crop`, `nitrogen`, `phosphorus`, `potassium`, `ph`. The endpoint requires `temperature`, `humidity`, `moisture` and `soil_type` as well, and does not accept `ph`. A request built from the current form fails validation with 422. Four new inputs must be added — no conversion from pH exists or should be invented.

B6 — Endpoint paths and port do not match HIGH

Three of four configured paths point at routes that do not exist, and `API_BASE_URL` uses port 8000 while `backend/app.py` documents port 5000.

B7 — Transaction type enums are entirely incompatible HIGH

No frontend `type` value equals any backend `entry_type` value, and the frontend's `expense` category has no backend representation at all.

B8 — The database is empty and its path is fragile HIGH

The database is gitignored, so a fresh clone starts with zero rows and every finance screen would legitimately render empty. Separately, `DB_PATH` is relative in both `backend/ledger.py` and `backend/models/user.py`: starting `uvicorn` from any directory other than the repository root silently creates a *new, empty* database rather than failing loudly.

B9 — Response scales differ silently MEDIUM

`confidence` and `risk_score` are 0–1 floats from the backend and 0–100 integers in the frontend. This will not raise an error — it will render “1% match” for a 94%-confident prediction. Silent numeric bugs are more dangerous than crashes.

13. Recommended Integration Order

Derived from the actual dependencies in this repository, not a generic template.

#	STEP	WHY THIS POSITION	EFFORT
0	Commit the crop model artifacts (adjust <code>.gitignore</code> or <code>git add -f</code>)	Nothing about crop can be tested until a fresh clone has them	Trivial
1	Fix <code>DB_PATH</code> to resolve from <code>backend/</code>	Prevents an entire class of “why is it empty?” debugging	Trivial
2	Build the frontend HTTP layer	B1 — every later step depends on it	Small

#	STEP	WHY THIS POSITION	EFFORT
3	Build the auth layer (login, token store, Bearer, 401 retry)	B2 — six endpoints are gated	Medium
4	Fix <code>Config.ENDPOINTS</code> paths and the port	B6 — cheap, and blocks everything otherwise	Trivial
5	Wire Crop Recommendation	Simplest real contract: 3 renames + one ×100	Small
6	Add the missing fertilizer inputs, then wire it	B5 — needs a form change first	Medium
7	Wire the Ledger (list + verify)	Highest demo value; the chain is already real	Medium
8	Add members / transactions / loans / summary endpoints	B4 — new backend work; unblocks four screens at once	Large
9	Wire Savings writes to <code>POST /ledger/add</code>	Makes the app genuinely persistent	Small
10	Wire Loan Risk	Depends on auth (3) and members (8)	Medium
11	Rebuild Home from the working services	Depends on almost everything above	Small
12	Re-point the Assistant at live services	Depends on the services being real	Small
13	Decide the fate of Crop Health / My Land / Reports	Product decision, not an engineering one	TBD

Steps 0, 1 and 4 are trivial and unblock disproportionate amounts of later work — do them first.

14. Recommended Future Sources Of Truth

Architecture guidance only. **Not implemented in this task.**

DATA	FUTURE SOURCE OF TRUTH	NOTES
Members (identity)	<code>users</code> table	Needs <code>phone</code> , <code>village</code> , <code>joined_at</code> columns added
Member savings	Backend aggregation over <code>ledger</code>	Never a stored column — keep it derived
Outstanding loans	Backend aggregation over <code>ledger</code>	Same
Group corpus / summary	Backend aggregation	One endpoint, so Home and Finance cannot disagree
Transactions	<code>ledger</code> table	The ledger is the transaction log — do not add a second table
Loans	<code>ledger</code> , or a new <code>loans</code> table if term and purpose must persist	Product decision
Loan risk	ML model + ledger-derived savings consistency	Never computed in the frontend
Ledger verification	<code>backend/ledger.py::verify_chain()</code>	The UI must never be able to show VERIFIED on its own
Crop recommendation	Trained crop model via <code>/predict-crop</code>	Latest result held in frontend session state for display only
Fertilizer advice	Trained fertilizer model + guideline table	Keep the <code>source</code> tag visible
Assistant answers	Composed at query time from the above	No second copy of any figure
Crop Health	TBD — no backend exists	Descope, or design a table
My Land	TBD — no backend exists	Descope, or design a table
Reports	Backend aggregation for the fund half	Field half blocked on the two above
Group name / branding	Frontend config	Genuinely presentation, not data

15. Final Summary

The backend is in better shape than the frontend's readiness to use it. The FastAPI service has real authentication, a real tamper-evident ledger and two working ML pipelines, and several of its best capabilities — the crop→fertilizer advisory chain, ledger-derived savings consistency, ranked crop alternatives — are not surfaced anywhere in the app.

The frontend is complete and polished as a **prototype**, but it was built against mocks and has no HTTP or auth layer at all. The gap is therefore mostly *frontend wiring plus a handful of new endpoints*, rather than a deep architectural conflict.

Three corrections to expectations set before this analysis

1. **The crop form does not need a farmer-friendly conversion.** On this branch it already collects the seven technical values the model expects. Crop is one of the easiest integrations, not one of the hardest.
2. **The frontend's mocks are not duplicated.** They cross-import from single sources. The duplication risk is across the frontend/backend boundary only.
3. **Fertilizer, not crop, is the worst mismatch.** The form is missing four *required* API fields and displays three values the model never produces.

Highest-value quick wins: commit the crop artifacts, fix `DB_PATH`, and correct the four endpoint paths. All three are trivial and each unblocks work that is otherwise impossible to test.

16. Next Stage — Hardware + Software Integration

Informational only. No hardware work was designed, started or implemented in this task.

Once the frontend/backend contracts above are aligned, the next project stage will combine the hardware and software sides of SahAI.

Current position

- The repository contains **no** `hardware/` **directory** and no embedded, ESP32 or IoT code of any kind.
- No API endpoint currently accepts sensor input.
- No database table currently stores device readings.

Two observations to carry into that stage

Stated as context, not as a design:

1. Several values the ML models require are exactly the kind a field sensor measures — soil `moisture`, `temperature` and `humidity` are all required by `/recommend-fertilizer` and are currently the fields the form cannot supply (§8.2).
2. The ledger's hash-chain design is append-only and already validates every write, so any future device-sourced record would need the same treatment as a human-entered one.

Detailed hardware/software design should begin only after this report is reviewed and the contract gaps above are resolved.

Prepared from a direct read of `secure-connect-prototype @ 3e34621`. Where the repository does not answer a question, this report says so explicitly rather than inferring. No code was modified, committed or pushed.